BodySculptApp – Project 4: Full System Test & Deployment Document

Course: CSCE 548
Student: Myra Walker
Semester: Spring 2026

1. Project Overview

The **BodySculptApp** is a full-stack application designed to help users track their workouts and diet information to support fitness and body sculpting goals. The system allows users to create, update, and retrieve workout and meal data, including detailed entries such as exercises performed and nutritional information.

This project demonstrates a complete **n-tier architecture**, consisting of:

- Database Layer (PostgreSQL)
- Data Access Layer (DAL)
- Business Layer
- Service/API Layer
- Client/Frontend Layer

The application was developed using **C++**, with a web-based frontend built using **HTML, CSS, and JavaScript**.

2. Technology Stack

- **Programming Language:** C++
- **Database:** PostgreSQL
- **Database Tool:** pgAdmin
- **Backend Framework:** Custom HTTP Server using cpp-httplib
- **Frontend:** HTML, CSS, JavaScript
- **IDE:** Visual Studio Community 2026
- **Version Control:** GitHub
- **Logging:** Custom Logger (C++)

3. System Architecture

The application follows a layered architecture:

1. Database Layer

Stores all persistent data using PostgreSQL tables:

- users
- meals
- meal_items
- workouts
- workout_entries

2. Data Access Layer (DAL)

Handles all direct database operations:

- SQL queries
- CRUD operations
- Database connections

3. Business Layer

Implements application logic:

- Validates data
- Coordinates operations between layers
- Calls DAL methods

4. Service Layer (API)

- Hosted via HTTP server
- Exposes endpoints for frontend interaction
- Handles requests and responses

5. Client Layer (Frontend)

- Web interface (HTML/JS)

- Sends requests to API
- Displays results to user

4. Database Schema Summary

The database contains five main tables:

users

- user_id (PK)
- first_name
- last_name
- email
- height_cm
- created_at

meals

- meal_id (PK)
- user_id (FK)
- meal_time
- meal_type
- notes

meal_items

- item_id (PK)
- meal_id (FK)
- food_name
- calories
- protein_g
- carbs_g
- fat_g

workouts

- workout_id (PK)
- user_id (FK)
- workout_date
- duration_min
- notes

workout_entries

- entry_id (PK)
- workout_id (FK)
- exercise_name
- sets
- reps
- weight_kg

Relationships:

- Users → Meals (1:M)
- Users → Workouts (1:M)
- Meals → MealItems (1:M)
- Workouts → WorkoutEntries (1:M)

5. Full System Test

The system was tested end-to-end to verify full functionality across all layers:

Client → API → Business Layer → DAL → Database → back to Client

Test 1: Create (Insert) Operation

Steps:

1. Start backend server

2. Open frontend
3. Enter user data and submit

Expected Result:

- User appears in frontend
- New record inserted in database

Test 2: Update Operation

Steps:

1. Select existing user
2. Modify user data
3. Submit update request

Expected Result:

- Updated data shown in frontend
- Database reflects updated values

Test 3: Get All Records

Steps:

1. Click “Get All Users” (or equivalent)

Expected Result:

- All records displayed in frontend

Test 4: Get Single Record (By ID)

Steps:

1. Enter specific ID
2. Retrieve record

Expected Result:

- Correct record displayed

Test 5: End-to-End Data Flow Validation

This confirms:

- Frontend successfully communicates with API
- API processes request correctly
- Business layer logic executes properly
- DAL interacts with database correctly
- Database changes persist

6. Deployment Instructions

This section explains how to run the project from scratch.

Step 1: Clone Repository

Download or clone the repository from GitHub:

- Option 1: Download ZIP and extract
- Option 2: Use Git:

```
git clone <repo-url>
```

Step 2: Install Prerequisites

Install the following:

- Visual Studio Community 2026
- Visual Studio Community 2026 (with C++ workload)
- PostgreSQL
- pgAdmin
- Web browser (Chrome recommended)

Step 3: Setup Database

1. Open pgAdmin
2. Create a new database
3. Run SQL scripts:
    - Create tables
    - Insert test data
4. Verify tables exist:
    - users
    - meals
    - meal_items
    - workouts
    - workout_entries

Step 4: Configure Database Connection

Update database credentials in code (Db.h or equivalent):

- database name
- username
- password
- port (default 5432)

Step 5: Build the Project

1. Open BodySculptApp.slnx file in Visual Studio Community 2026
    - Relative Path: "..\GitHub\CSCE-548-Project\BodySculptApp\BodySculptApp.slnx"
2. Build solution:
    - Build → Build Solution

Step 6: Run Backend Server

1. Run the project
    1. Standard Toolbar → Local Windows Debugger (Solid Green Play Button)
    2. Backend console appears as a popup. Minimize the popup, but DO NOT EXIT it (the server will shut down).
2. Confirm server starts:
    1. Open browser (Chrome recommended)
    2. In the search bar, Enter:
        - localhost:8080/health
    3. Expected output:
        - {"success":true,"message":"Server is running"}

        Backend is now running successfully!

Step 7: Run Frontend Web Application

1. Open browser (Chrome recommended)
2. In the search bar, Enter:
    - http://localhost:8080/ui/index.html
3. Expected output:
    - Body Sculpt App web application opens in browser

    Frontend is now running successfully!

Step 8: Verify Application Works

Test:

- Create user
- Update user

- Retrieve records

Confirm:

- No errors in console
- Database updates correctly
- Frontend displays data

Step 9: Troubleshooting

Common issues:

- **Server not starting**
    - Check port conflicts
- **Database connection failure**
    - Verify credentials
- **Frontend not working**
    - Ensure backend is running
- **Data not updating**
    - Check API routes and logs

7. AI Tool Usage and Reflection

Prompt Used for Project 4

The following prompt was used to generate code and structure for Project 4:

In previous chats labeled: Branch · Full Stack - CSCE 548 – Project, CSCE 548 - Project 2 Development, and CSCE 548 - Project 3 Web Client, you helped me complete phase 1-3 of my project. The instructions for phase 1 - 2 are included in attached images.

These are the instructions I gave you to help me complete phase 1:

Help me complete the first phase of my semester long project with step by step instructions. C++ will be the programming language of choice. My project idea is a body

sculpting app that keeps track users' workout and diet details in order to help them sculpt the body of their dreams.

These are the instructions I gave you to help me complete phase 2:

Now, I want you to help me complete phase 2 of my project. I would also like to add the optional logging functionality. The instructions, ERD, and solution explorer (showing how the app's files are organized) are also included in the attached images.

These are the instructions I gave you to help me complete phase 3:

Now, I want you to help me complete phase 3 of my project. The instructions for all of the different phases, ERD, and solution explorer (showing how the app's files are organized) are also included in the attached images.

Instructions for phase 4:

Now, I want you to help me complete phase 4 of my project. The instructions for all of the different phases, ERD, and solution explorer (showing how the app's files are organized) are also included in the attached images.

Changes Made to AI-Generated Code

- Fixed missing CRUD methods
- Corrected database queries
- Adjusted project structure
- Improved API endpoints
- Fixed frontend-backend communication
- Resolved build and configuration issues
- Added logging functionality
- Verified correct data flow across layers

Effectiveness of AI Tool

The AI tool was highly effective in:

- Generating boilerplate code

- Structuring application layers
- Providing guidance for implementation
- Accelerating development

However, manual work was required to:

- Debug errors
- Integrate components
- Validate data flow
- Fix missing functionality

Errors Encountered

- Database connection issues
- Missing CRUD operations
- Frontend not calling correct endpoints
- Build configuration problems
- File path issues for frontend

Conclusion

AI significantly improved development speed but required manual refinement to ensure correctness and full system integration.

8. Conclusion

The BodySculptApp successfully demonstrates a complete full-stack application using a layered architecture. All required functionality has been implemented and tested, including full system interaction from frontend to database.

The project meets all requirements for Project 4, including:

- Full system testing
- Deployment documentation

- Functional frontend and backend integration
- GitHub repository submission